Information Retrieval and Generative Artificial Intelligence

Term Project I
Web Search Engine

Due November/12/2026

1 Goal and scope

Build a small-scale, topic-focused web search engine. Collect public web pages, preprocess their text, build an inverted index, and retrieve ranked results through a simple web interface. The required system uses traditional information retrieval; LLMs, embeddings, RAG, and generative answers are not required.

- Understand how a crawler discovers pages and obeys crawling limits.
- Implement an inverted index, TF-IDF weighting, and cosine-similarity ranking.
- Evaluate search quality using manually judged relevance and Precision@5.

2 Web spider

- Choose a focused topic and 5–10 seed URLs. Declare an explicit allowlist of hostnames; subdomains must be listed if used. Follow links only within that scope, including redirects.
- Collect 200–1,000 valid pages with nonempty main text. Count unique normalized URLs, not failed requests. If the chosen sites cannot provide 200 suitable pages, revise the seeds.
- Crawl only public HTTP/HTTPS HTML pages. Exclude login pages, CAPTCHA-protected pages, downloads, and pages that require JavaScript rendering. Do not bypass access restrictions.
- Respect robots.txt and use an identifiable User-Agent. Keep at least two seconds between requests to the same host, including concurrent requests; honor longer site-requested delays.
- Set a request timeout (e.g., 10 seconds) and a total page-fetch attempt limit (e.g., 3,000, including failures). Avoid repeated retries; stop or back off on rate-limit responses. Log failures and skipped URLs.
- Resolve relative URLs and remove #fragments before checking the visited set. Advanced content deduplication is optional. Do not blindly remove query parameters, which may identify different pages. (e.g., https://ex.com/course.html and https://ex.com/course.html#grading are the same page)

- Extract URL, title, main text, and fetch time. Remove scripts, styles, and obvious navigation text where practical. Store the collection in JSON or SQLite using fields {url, title, text, fetch_time}.

3 Text processing and indexing

- Use English pages for the simplest implementation. Chinese or mixed-language collections are allowed if the report explains the Chinese segmentation method and language-specific preprocessing.
- Apply consistent preprocessing to documents and queries. Lowercase where appropriate, tokenize, and document the stopword policy. Handle punctuation carefully: avoid merging words such as machine-learning into machinelearning, and explain treatment of terms such as C++.
- Assign document IDs and build an inverted index mapping each term to postings containing document IDs and term frequencies. Store document frequencies and document-vector norms needed for ranking.
- Implement TF-IDF and state the exact TF and IDF definitions, including any smoothing or normalization. Use the same collection-derived vocabulary and IDF values for documents and queries; never fit a new IDF model on each query.

4 Search and web interface

- Accept a keyword query. Use the inverted index to identify candidate documents containing query terms, then rank them by cosine similarity between TF-IDF vectors.
- Return up to 10 results with positive similarity, ordered by score. Handle empty queries, and out-of-vocabulary queries without errors; show a clear message when no results match.
- Provide a simple Flask search box and results page. Each result must show its title, URL, snippet for summary, and score. Include the number of indexed pages.
- Use a plain-text snippet of approximately 50–100 English words or 80–150 Chinese characters, or the available text if shorter.
- Restrict result links to HTTP/HTTPS URLs. Run the demonstration locally; public deployment is not required.

5 Implementation boundaries

You may use libraries for HTTP requests, HTML parsing, Chinese segmentation, SQLite access, numerical operations, and Flask. You must implement the inverted index, TF-IDF weighting, and cosine-similarity ranking yourselves and use your index during retrieval.

Do not substitute a hosted search API or a ready-made search engine for the required components. TfidfVectorizer and similar end-to-end TF-IDF implementations may be used only to check your calculations, not to replace the required implementation. Be prepared to explain your code and demonstrate the scoring of a small example.

6 Required search quality evaluation

Precision@5 is a required component, not a bonus. Evaluate a fixed snapshot of your indexed collection so that the reported results can be reproduced.

- Prepare 10 distinct, topic-relevant test queries representing plausible user needs. Freeze the queries before the final evaluation; do not select only queries that produce good results.
- Write a short information need for each query and a relevance rule. Label each returned top-five document as relevant (1) or not relevant (0), based on its content rather than only its title or snippet.
- Save the query, rank, document ID or URL, relevance label, and a brief judgment reason in an evaluation file. Have another group member review ambiguous judgments when possible.
- Compute Precision@5 for each query, then report the arithmetic mean across all 10 queries. When fewer than five results are returned, treat missing positions as nonrelevant; the denominator remains five.

Precision@5 = number of relevant documents in the first five positions / 5

Example: three relevant documents among the first five give Precision@5 = 3/5 = 0.60. If only two results are returned and both are relevant, Precision@5 = 2/5 = 0.40.

- Analyze at least two weak or unsuccessful queries. Explain likely causes, such as missing vocabulary, poor extraction, tokenization, or ranking limitations, and suggest improvements.

Your evaluation is graded on its correctness, reproducibility, and analysis. A high Precision@5 alone does not guarantee a high grade, because topics and collections differ.

Optional understanding of Recall and F1

Recall is the number of relevant retrieved documents divided by the total number of relevant documents in a specified evaluation collection. Labeling only the returned results cannot establish this denominator. Recall does not refer to all relevant pages on the entire web.

Recall and F1 are not required. If reported, use a bounded evaluation collection with complete relevance judgments and state the retrieval cutoff. Compute precision and recall for the same retrieved set.

F1 = 2 × precision × recall / (precision + recall)

If precision and recall are both zero, report F1 as zero. If no relevant documents exist in the evaluation collection for a query, state how the undefined recall is handled.

7 Deliverables

- Working source code: crawler.py, indexer.py, search.py, app.py, and any supporting files. Equivalent modular organization is acceptable if documented.
- README with installation and run commands, dependency versions, seed URLs, allowed hosts, preprocessing choices, and steps to reproduce the evaluation.
- Dataset in JSON or SQLite, saved index files, crawling summary/logs, and the query and relevance-judgment file. Submit only the public page content needed for this assignment; do not include credentials or session cookies.

- The main group report must include: an architecture diagram; crawling statistics (attempts, valid pages, hosts, failures, and average text size); preprocessing and scoring definitions; 3–5 illustrated search examples; evaluation results for all 10 queries; analysis of at least two weak queries; implementation challenges; each member's contribution.
- Each student must separately submit his/her own reflections about what they learned and the difficulties they faced in this project.
- Upon demo, each group has to present the process of how they develop their search engine, a 3–5 minute live demonstration showing the search engine, representative queries, and an empty or no-match query, and what challenge they encountered and the reflections about what they learned from this project.

8 Evaluation rubric

Component	Points	Key criteria
Web spider	20	Correct scoped crawling, robots.txt, rate limits, useful stored text, and logs
Indexing and preprocessing	25	Appropriate tokenization, correct inverted index and documented TF-IDF
Query and ranking	25	Actual index use, correct cosine ranking, top-K results, and edge cases
Search quality evaluation	15	Fixed queries, relevance judgments, correct Precision@5, and failure analysis
Web interface	5	Usable local interface and safe text rendering
Report and demo	10	Clear explanation, reproducibility, individual reflections, and working demo
Total	**100**	

9 Optional bonus up to 10 points

Bonus is awarded for completed, explained features beyond the required system: BM25 comparison using the same queries and judgments (up to 5); phrase search with positions or verified contiguous matches, or Boolean queries (up to 3); query-time measurement with a clear timing boundary and summary (up to 2). Bonus does not replace required components. Maximum total: 110 points.